

Módulo 11: Concurrency y rendimiento

Contenidos clave

GIL y sus implicaciones: GIL - Global Interpreter Lock, es un mecanismo de Python que, en el intérprete CPython tradicional, hace que solo un thread ejecute código Python a la vez dentro de un proceso. Usar muchos threads no necesariamente acelera los cálculos pesados de CPU. Los threads son muy útiles cuando el programa pasa mucho tiempo esperando:

- una API
- una base de datos
- un archivo
- una conexión de red

Mientras un thread espera, otro puede avanzar.

threading y concurrent.futures: Unthread es un hilo de ejecución dentro de un proceso. Se usan principalmente para tareas I/O-bound: API -> esperar respuesta -> otra API -> esperar respuesta. Los threads permiten aprovechar ese tiempo de espera.

asyncio: event loop, async/await:

- asyncio: permite manejar muchas tareas de entrada/salida de manera concurrente utilizando un event loop (administra las tareas). En lugar de crear un thread para cada tarea, tenemos un ciclo que administra las tareas pendientes.
- async: indica que una función es asíncrona.
- await: indica "necesito esperar esta operación, pero mientras tanto permite que otras tareas avancen".

multiprocessing para CPU-bound: Si se tiene una tarea que requiere mucho procesamiento de CPU los threads están limitados por el GIL; para estos casos se pueden usar procesos independientes. Cada proceso tiene su propio intérprete de Python y su propio GIL.

Medición con timeit y cProfile:

- timeit: sirve para medir cuánto tarda un pequeño fragmento de código. Es útil para comparar.
- cProfile: sirve para analizar dónde está gastando tiempo nuestro programa. Esto permite detectar funciones que están consumiendo mucho tiempo.

Objetivos

- Elegir el modelo de concurrencia apropiado.

Según el tipo de tarea se usaron distintos modelos:

- asyncio y http.AsyncClient para operaciones de E/S, como solicitudes HTTP concurrentes.
- asyncio.Semaphore para limitar el número de solicitudes simultáneas y evitar sobrecargar el servicio.
- ProcessPoolExecutor para tareas intensivas de CPU, evitando las limitaciones del GIL.

- `timeit` y `cProfile` para medir tiempos y analizar el rendimiento

- Implementar E/S concurrente y medir mejoras.

Se implementaron versiones:

- Síncrona con `http.Client`.
- Asíncrona con `http.AsyncClient`.
- Concurrente mediante `asyncio.gather`.
- Limitada mediante un semáforo configurable.

Laboratorio

- Fetcher concurrente con `httpx.AsyncClient` y semáforo.

El fetcher se creó con `httpx.AsyncClient`, `asyncio.gather` y `asyncio.Semaphore` y garantiza que no se supere el límite de solicitudes concurrentes configurado.

- Comparación con versión síncrona.

Se compararon ambas implementaciones usando tiempos simulados y mocks reproducibles, evitando depender de Internet real. La versión asíncrona obtuvo mejores tiempos al ejecutar varias esperas de E/S en paralelo.

- Pequeño cálculo CPU-bound con `ProcessPoolExecutor`.

Se implementó `sum_of_squares` y su ejecución paralela con: `ProcessPoolExecutor`.

Para permitir la distribución del cálculo entre varios procesos y evitar las restricciones del GIL para tareas CPU.

Medición y validación:

Se utilizaron:

- `timeit` para medir tiempos.
- `cProfile` para perfilar la ejecución.
- `pytest` y mocks para validar el comportamiento.
- Inyección de `httpx.MockTransport` para pruebas reproducibles.
- Cobertura del 100% sobre `fetcher.py` y `cpu_tasks.py`.

```
> poetry run pytest .\labs\day11_concurrency\tests --cov=labs.day11_concurrency --cov-report=
term-missing --cov-report=html --cov-fail-under=100
===== test session starts =====
platform win32 -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Quiryat\Documents\Python\python-mastery-capstone
configfile: pyproject.toml
plugins: anyio-4.15.1, hypothesis-6.168.1, asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_defaul
t_test_loop_scope=function
collected 6 items

labs\day11_concurrency\tests\test_benchmark.py .    [ 16%]
labs\day11_concurrency\tests\test_concurrency.py ... [ 66%]
..                                                  [100%]

===== tests coverage =====
_____ coverage: platform win32, python 3.14.6-final-0 _____

Name                                                    Stmts  Miss  Cover   Missing
-----
labs\day11_concurrency\cpu_tasks.py                     9      0   100%
labs\day11_concurrency\fetcher.py                      23      0   100%
-----
TOTAL                                                    32      0   100%
Coverage HTML written to dir htmlcov
Required test coverage of 100% reached. Total coverage: 100.00%
===== 6 passed in 1.66s =====
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone
>
```